

속성 기반 제약 생성으로 의미 지키기

박준성, 김진상, 이우석
한양대학교 프로그래밍시스템 연구실 (PSL)



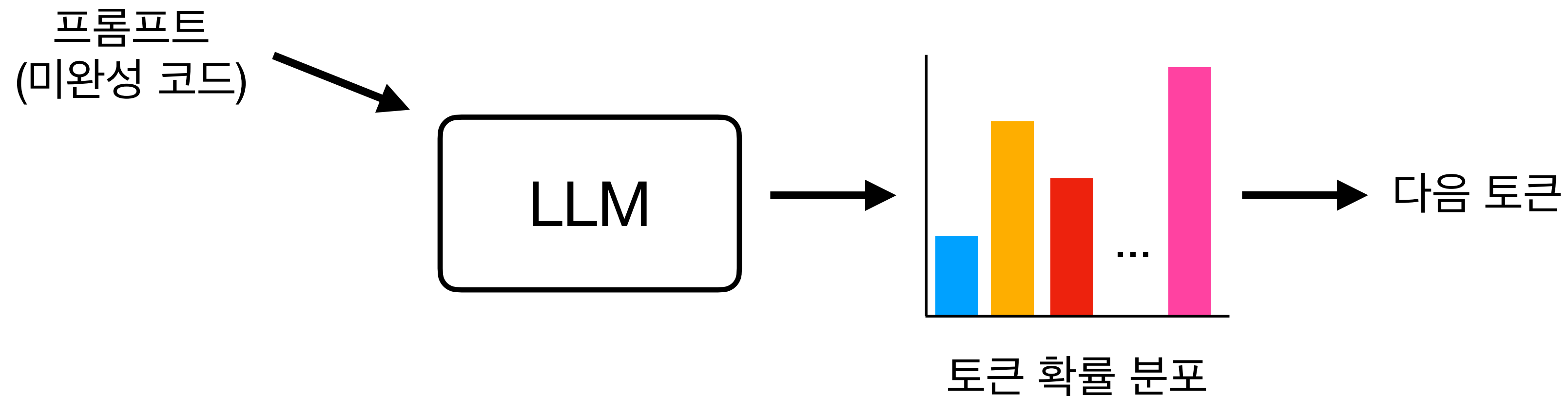
배경 지식

제약 생성(Constrained decoding)이란?

배경 지식

제약 생성(Constrained decoding)이란?

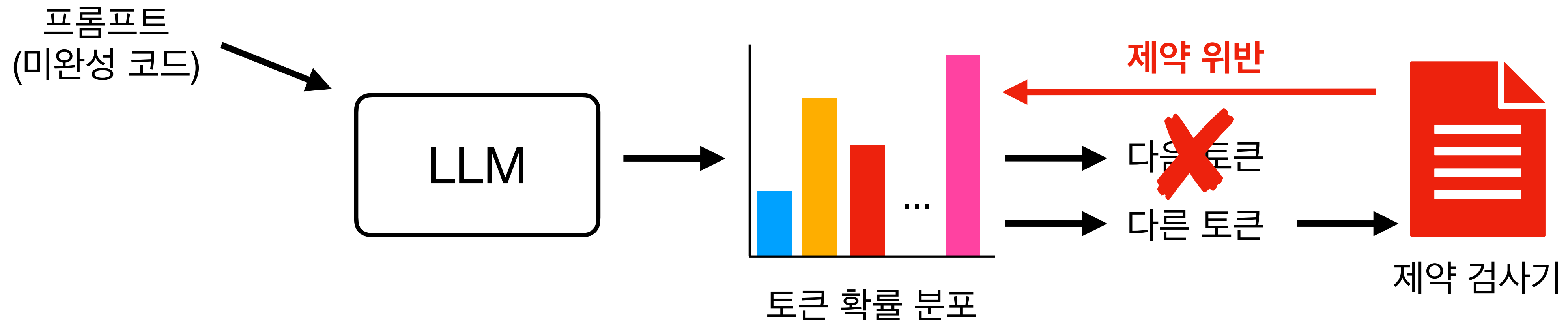
- LLM의 코드 생성 과정 중
특정 규칙이나 제약을 만족하도록 다음 토큰 선택을 제한하는 방법



배경 지식

제약 생성(Constrained decoding)이란?

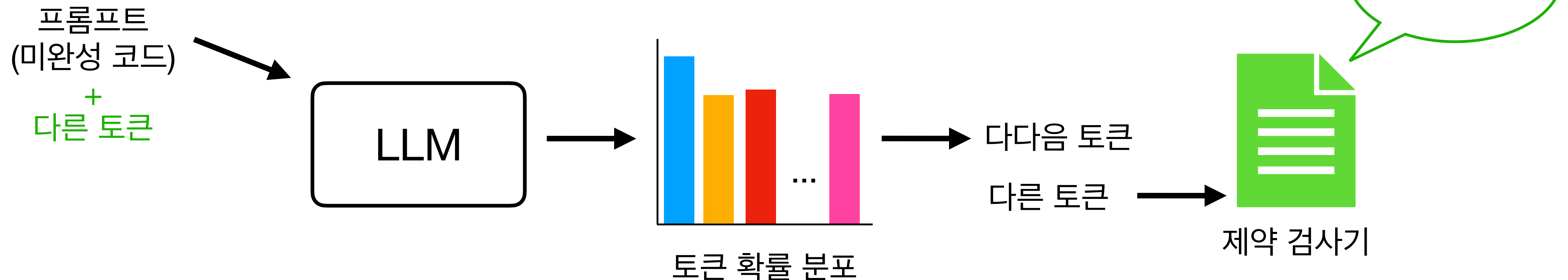
- LLM의 코드 생성 과정 중
특정 규칙이나 제약을 만족하도록 다음 토큰 선택을 제한하는 방법



배경 지식

제약 생성(Constrained decoding)이란?

- LLM의 코드 생성 과정 중
특정 규칙이나 제약을 만족하도록 다음 토큰 선택을 제한하는 방법



배경 지식

기존 방법의 한계

- 대부분의 제약 생성 방법론이 문법적 올바름만 보장

배경 지식

기존 방법의 한계

- 대부분의 제약 생성 방법론이 문법적 올바름만 보장
- 그 외 의미적 올바름을 보장하는 기존 제약 생성 방법은
 1. 매 토큰 생성 단계마다 지금까지 생성한 코드 전부를 분석
 2. 목표 언어와 특성을 위한 분석기를 직접 만듦

배경 지식

기존 방법의 한계

- 대부분의 제약 생성 방법론이 문법적 올바름만 보장
- 그 외 의미적 올바름을 보장하는 기존 제약 생성 방법은
 1. 매 토큰 생성 단계마다 지금까지 생성한 코드 전부를 분석
분석이 언제, 어디에 필요한지는 속성을 통해 판단하고,
 2. 목표 언어와 특성을 위한 분석기를 직접 만듦
기존의 분석기를 활용하여 의미적 올바름 보장

속성 기반 제약 생성!

분석기만 사용하면 안돼?

알아보기

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.

분석기만 사용하면 안돼?

알아보기

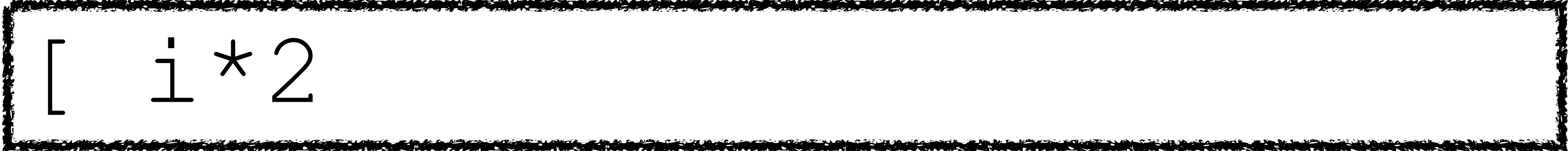
- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.

```
[ i * 2
```

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.



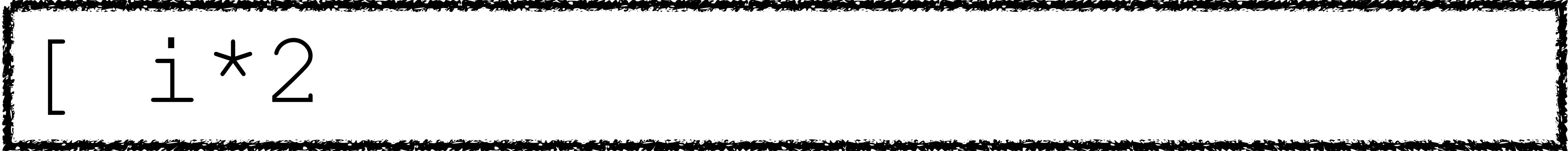
```
[ i * 2
```

- 분석기: 이거 변수명 오류 있는데? `i`는 정의된 변수명이 아니야.

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.



```
[ i * 2
```

- 분석기: 이거 변수명 오류 있는데? `i`는 정의된 변수명이 아니야. → 코드 생성 ✗

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.

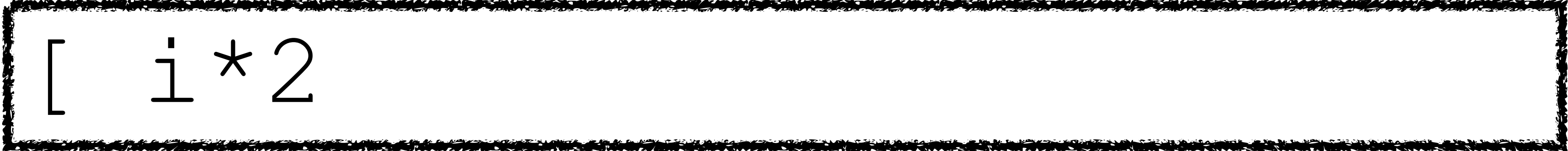
```
[ i*2 for i in range(10) ]
```

- 분석기: 이거 변수명 오류 있는데? `i`는 정의된 변수명이 아니야.
 - ... 하지만 나중에 `for`문이 온다면 변수가 정의될 수 있음. (즉, 분석기 때문에 과하게 거부함)

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.



```
[ i * 2
```

- 임시 방편:

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.



```
[ i * 2
```

괄호가 열리는 게 문제인가...?

- 임시 방편:

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.



```
[ i * 2
```

괄호가 열리는 게 문제인가...?

- 임시 방편: 그럼 괄호가 열려 있으면 검사를 미루다가 닫힐 때 분석기를 부를까...?

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.

```
[ i*2 for i in
```

- 분석기: 괄호가 열려 있으니까 나중에 검사 해야겠다.

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.

```
[ i*2 for i in X
```

- 분석기: 괄호가 열려 있으니까 나중에 검사 해야겠다.

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.

```
[ i*2 for i in X
```

이건 나중에 검사하면 안됨

- 분석기: 괄호가 열려 있으니까 나중에 검사 해야겠다.

분석기만 사용하면 안돼?

알아보기

- 코드 앞 부분만 보고는 올바른 판단을 위한 맥락이 부족한 경우가 존재
- 이를 테면, 변수명 오류 분석기와 함께 파이썬 코드를 제약 생성하는 상황을 생각해보자.

```
[ i*2 for i in X
```

- 분석기: 괄호가 열려 있으니까 나중에 검사 해야겠다.

이건 나중에 검사하면 안됨
for문이 새 변수명을 정의해도
여기선 잡히지 않음

분석기만 사용하면 안돼?

임시 방편은 어디까지나 임시 방편

- 방금 예제처럼 특정 케이스마다 딱 맞는 규칙ad-hoc rule을 추가하는 건 확장성이 떨어짐

분석기만 사용하면 안돼?

임시 방편은 어디까지나 임시 방편

- 방금 예제처럼 특정 케이스마다 딱 맞는 규칙ad-hoc rule을 추가하는 건 확장성이 떨어짐
- 이처럼 언제 검사를 해야 할까에 대한 판단은 보통 문법에 의존적임

분석기만 사용하면 안돼?

임시 방편은 어디까지나 임시 방편

- 방금 예제처럼 특정 케이스마다 딱 맞는 규칙ad-hoc rule을 추가하는 건 확장성이 떨어짐
- 이처럼 언제 검사를 해야 할까에 대한 판단은 보통 문법에 의존적임
- 따라서 문법 위에 의미 규칙을 얹은 **속성 문법**을 이용해보자!

그럼 속성만 사용하면 안돼?

예제: 타입 오류 방지하기

그럼 속성만 사용하면 안돼?

예제: 타입 오류 방지하기

- 코드를 문법에 따라 파싱하면서
 - 현재 사용 가능한 변수와 그 변수의 타입 관리
 - 지금 만들고 있는 표현식이 최종적으로 되어야 하는 타입
 - ...
- 을 계산

그럼 속성만 사용하면 안돼?

예제: 타입 오류 방지하기

- ...을! 이미 했던 기존 연구가 있음
 - Type Constrained Code Generation (PLDI 25') : 이하 TCG
 - 약 7000 줄의 코드로 수제 TypeScript 파서 제작

그럼 속성만 사용하면 안돼?

TCG가 생성한 코드

```
function add_lists(test_list: number[], test_tup: [number, number]):  
[number, number, number, number, number] {  
    const result = test_list.concat(test_tup[0] as number[]);  
    result.push(test_tup[1]);  
    return result as [number, number, number, number, number];  
}
```

그럼 속성만 사용하면 안돼?

TCG가 생성한 코드

```
function add_lists(test_list: number[], test_tup: [number, number]):  
[number, number, number, number, number] {  
    const result = test_list.concat(test_tup[0] as number[]);  
    result.push(test_tup[1]);  
    return result as [number, number, number, number, number];  
}
```

- 타입 오류 발생: number를 number[]로 캐스팅하려 했음

그럼 속성만 사용하면 안돼?

속성은 어디까지나 속성

- 알고보니 **누락된** 타입 규칙이 존재

그럼 속성만 사용하면 안돼?

속성은 어디까지나 속성

- 알고보니 누락된 타입 규칙이 존재
- 그럼 다음 할 일: 누락된 의미 규칙 추가

그럼 속성만 사용하면 안돼?

속성은 어디까지나 속성

- 알고보니 누락된 타입 규칙이 존재
- 그럼 다음 할 일: 누락된 의미 규칙 추가
- 계속 의미 규칙을 **추가, 추가, 추가, 추가** ...

그럼 속성만 사용하면 안돼?

속성은 어디까지나 속성

- 알고보니 누락된 타입 규칙이 존재
- 그럼 다음 할 일: 누락된 의미 규칙 추가
- 계속 의미 규칙을 추가, 추가, 추가, 추가 ...
- 그럼 사실상 **Typescript 컴파일러**를 만드는 일: **201K LoC**

그럼 속성만 사용하면 안돼?

속성은 어디까지나 속성

- 알고보니 누락된 타입 규칙이 존재
- 그럼 다음 할 일: 누락된 의미 규칙 추가
- 계속 의미 규칙을 추가, 추가, 추가, 추가 ...
- 그럼 사실상 Typescript 컴파일러를 만드는 일: 201K LoC

이미 존재하는 분석기를 파서로 옮기는 꼴

그럼 속성만 사용하면 안돼?

속성은 어디까지나 속성

- 알고보니 누락된 타입 규칙이 존재
- 그럼 다음 할 일: 누락된 의미 규칙 추가
- 계속 의미 규칙을 추가, 추가, 추가, 추가 ...
- 그럼 사실상 Typescript 컴파일러를 만드는 일: 201K LoC

이미 존재하는 분석기를 파서로 옮기는 꼴

속성을 최소한으로 분석기를 언제/어디에 활용할지만을 정하는 용도로만

Poster

• 포스터에서는...

• OCaml/Python 변수명 오용 방지

• TypeScript 타입 오류 방지

언제 분석할까?: 속성 기반 제약 생성으로 의미 지키기

When to Analyze: Attribute-Guided Semantic Filtering for Constrained Decoding

박준성, 김진상, 이우석

한양대학교 프로그래밍 시스템 연구실 (PSL)



문 제

제약 생성(Constrained Decoding)



- LLM의 코드 생성 과정 중 특정 규칙이나 제약을 만족하도록 다음 토큰 선택을 제한하는 방법
- 대부분의 제약 생성 방법은 문법적 올바름만 보장
- 의미적 올바름 보장을 위해서는 미완성 코드에 대한 정적 분석 필요

기존 의미 제약 생성 방법의 한계

- 어떤 특성을 언제, 어떻게 분석할지가 하나로 뭉쳐져 있음
 - 매 토큰 생성마다 지금까지 만든 코드 전체 분석*
 - 목표 언어+특성을 위한 분석기 직접 만들*
- 속성 기반 제약 생성:
 - 분석이 언제, 어디에 필요한지는 속성을 통해 판단하고,
 - 기존의 분석기를 활용하여 의미적 올바름 보장

*Mindor et al. "Type-Constrained Code Generation with Language Models". PLDI 2022
*Zhang, Shuan, et al. "ChapChap: A Programmable Framework for Semantically Constraining the Outputs of Language Models". POPL 2023

속성 기반 제약 생성을 위한 준비물

- 속성 문법 (문법 + 의미 규칙)
 - 속성: 문법 기호에 저장되는 변수
 - 의미 규칙: 속성이 어떻게 변해야 하는지
- 점진적 파서
 - 새로 들어온 입력만 추가로 파싱하는 파서
- 기존 분석기
 - 어떻게 의미 검사를 수행할지 결정

개 요

분석기만 이용한다면?

- 코드 앞 부분만 보고는 올바름 판단을 위한 맥락이 부족한 경우 존재
- ex. 변수명 오류 분석기와 함께 파이썬 코드 제약 생성

```
[i for i in range(10)] [i for i in X nge(10)]
```

이 시점에서 검사하면 변수명 오류
그러나 뒤에 for문이 오면 정의 가능

'괄호가 열리면 검사 미루기'와 같이
케이스 별로 규칙을 추가하는 건 한계가 있음

문법에 따라
의미 규칙에 의해
계산되는 '속성'을 사용

속성만 이용한다면? ex. 타입 오류 방지하는 제약 생성하기

- 타입 환경을 구성하고 그에 맞는 코드만 허용하도록 문법에 따라 계산
- ...을 실제로 하고 있는 기존 연구(PLDI 25**)가 이미 있음
 - 누락된 의미 규칙으로 실수 허용: 10 as number[]
- 전체 의미 규칙을 지원하려면 Typescript 컴파일러(201K) 구현해야함
- 이미 있는 분석기 또 만들지 말고, 직접 활용하여 의미적 올바름을 보장

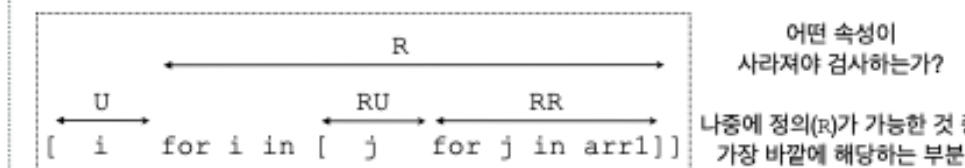
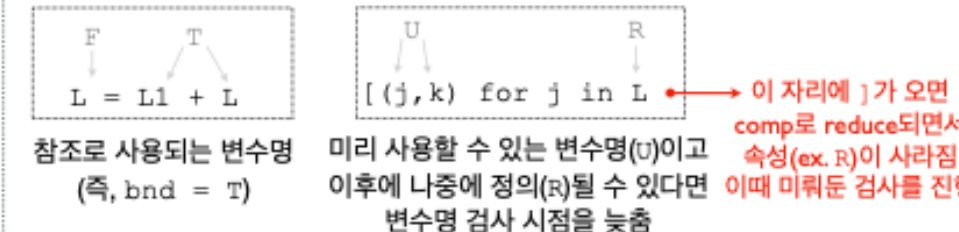
적 용

정의되지 않은 변수명 오류 방지

```
assign -> name "=" exp   name.bnd = F, exp.ctx = ε
exp    -> name             name.bnd = T, exp.ctx = name.ctx
      | "[" comp "]"       comp.ctx = exp.ctx
comp   -> exp fors         exp.ctx = comp.ctx ++ U,
                        fors.ctx = comp.ctx ++ R
```

- bnd : 참조로 사용되는 변수명인지
- ctx : 미리 사용(u)하고 나중에 정의(r)될 수 있는 맥락이 얼마나 쌓였는지

어떤 기호를 언제 분석할까?



기호를 어떻게 분석할까?

- 변수명이 코드 중간에 있는 경우: 가능한 변수명과 정확히 일치하는지 검사
- 코드 끝 부분에 있는 경우: 더 이어지면 가능한 변수명이 될 수 있는지 검사

```
def f(L1, L2):
  X = L1 + L
```

① $L1 \in \{L1, L2\} \rightarrow \checkmark$
② $L \in \text{prefix}(\{L1, L2\}) = \{L1, L2, L\} \rightarrow \checkmark$

실험 (5개 seed로 샘플링하여 합산)

- Python : 벤치마크 - BigCodeBench-Hard / 분석기 - Pyflakes + Jedi
- OCaml : 벤치마크 - MultiPL-E / 분석기 - Merlin
- Baseline : 변수명 오류를 피드백으로 주고 다시 생성 (최대 5회) / Timeout : 90s

Model	# Target Errors ↓			Per-token Time (ms) ↓			pass@1 ↑		
	VANILLA	Baseline*	GENCOR	VANILLA	Baseline*	GENCOR	VANILLA	Baseline*	GENCOR
Name -- Python									
CodeLlama 7B	105	54 [48.4%	36 [65.7%	26.2	70.7 [169.0%	40.2 [73.4%	17	18 [75.9%	18 [75.9%
DS-Coder 6.7B	30	8 [27.3%	4 [16.7%	26.4	46.3 [151.1%	46.1 [74.4%	59	59 [70.0%	59 [70.0%
Qwen2.5 7B	40	2 [46.7%	7 [18.3%	27.6	46.8 [142.0%	55.6 [101.4%	101	103 [72.0%	104 [73.0%
CodeLlama 34B	209	39 [18.3%	58 [27.2%	55.8	154.6 [177.1%	70.9 [127.1%	15	16 [76.7%	15 [70.0%
Name -- OCaml									
CodeLlama 7B	672	263 [39.0%	139 [20.3%	26.0	87.4 [126.2%	37.5 [74.2%	338	355 [75.0%	364 [77.2%
DS-Coder 6.7B	349	123 [34.8%	59 [16.3%	26.6	70.5 [165.0%	37.9 [74.3%	776	800 [73.1%	802 [73.4%
Qwen2.5 7B	461	134 [29.0%	67 [14.5%	27.3	68.1 [149.3%	46.4 [70.0%	968	1051 [78.6%	1029 [76.3%
CodeLlama 34B	674	197 [29.0%	150 [22.2%	54.9	158.7 [189.1%	65.3 [118.9%	386	421 [79.1%	413 [77.0%

타입 오류 방지

```
stmt -> LET IDENT ":" TYPE "=" exp ";" exp.xt = T
      | LET IDENT "=" exp ";" exp.xt = F
exp   -> IDENT
      | exp1 "(" exp2 ")" exp1.xt = F, exp2.xt = T
```

- xt = T : 기대 타입 O
 - xt = F : 기대 타입 X, 타입은 성립
- let x : number = <Hole>
let x = <Hole>

어떤 기호를 언제 분석할까?

- exp 기호는 항상 검사해야 함
 - 어떤 타입으로 성립 할 수 있나(xt = F)
 - 기대 타입으로 도달 가능한가(xt = T)

기호를 어떻게 분석할까?

```
let age = 7 + "years old"
```

타입 선언 없음 exp.xt = F

① TypeScript Compiler API에 미완성 표현식 위치의 현재 타입을 질의

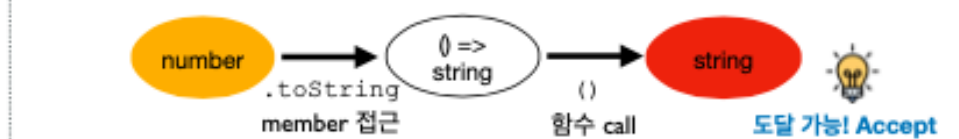
- 현재 타입: string("7 years old"), 조회만 되면 ✓

```
let num = 12.5
let int_num = parseInt(num)
```

함수 인자 exp2.xt = T

① TypeScript Compiler API에 미완성 표현식 위치의 기대 타입과 현재 타입을 질의

- 기대 타입: string(=parseInt의 첫번째 인자), 현재 타입: number(=num)
- 현재 타입이 기대 타입으로 확장될 수 있는가?
- 타입 도달성 그래프 탐색



실험 (5개 seed로 샘플링하여 합산)

- Typescript : 벤치마크 - MultiPL-E / 분석기 - Typescript Compiler API
- Baseline : type-constrained code generation (PLDI 25**) / Timeout: 90s

Model	# Target Errors ↓			Per-token Time (ms) ↓			pass@1 ↑		
	VANILLA	Baseline*	GENCOR	VANILLA	Baseline*	GENCOR	VANILLA	Baseline*	GENCOR
Type -- TypeScript									
CodeLlama 7B	309	174 [55.9%	184 [59.5%	25.4	46.8 [184.3%	70.7 [121.0%	1143	1190 [74.1%	1192 [74.3%
DS-Coder 6.7B	174	71 [40.2%	63 [36.8%	25.1	43.0 [71.3%	77.6 [100.2%	1771	1819 [103.0%	1824 [103.4%
Qwen2.5 7B	483	74 [14.7%	73 [14.9%	27.7	47.1 [70.0%	104.0 [127.5%	1714	1978 [115.4%	2005 [117.0%
CodeLlama 34B	502	224 [44.6%	274 [54.4%	54.1	72.5 [134.0%	96.5 [176.4%	1154	1238 [77.3%	1235 [77.0%